```
array_walk($cars, 'applySanity2', 'CHANGED_');
print "\n\nAfter applySanity2:\n";
print_r($cars);

function applySanity2 (&$val, &$key, $append)
{
    $key = $append  . $key;
    $val = [
        $val[0][0] => $val[0][1],
        $val[1][0] => $val[1][1],
    ];
}

Output:
Warning: applySanity2(): Argument #2 ($key) must be passed by reference,
value given in php shell code on line 7

After applySanity2:
Array
(
    [car1] => Array
        (
            [make] => VW
            [model] => Golf GTI
        )

)
```

Although `array_walk()` does its intended job with respect to values, the keys remain unchanged; our assignment to `$key` had no effect because it is treated as if it was passed by value. PHP also warns us that we're being naughty. Note that the error message is somewhat counter-intuitive!

Finally, although it would seem that passing the third argument to `applySanity()` by reference would be useful for building new data structures (and leaving the originals intact) this doesn't work, neither does it cause any errors. For that functionality, see the next example – `array_map()`.